

Documentation

Détection de personnes en réseau local

Yves Jahina Chekoua

1^{er} septembre 2026

Table des matières

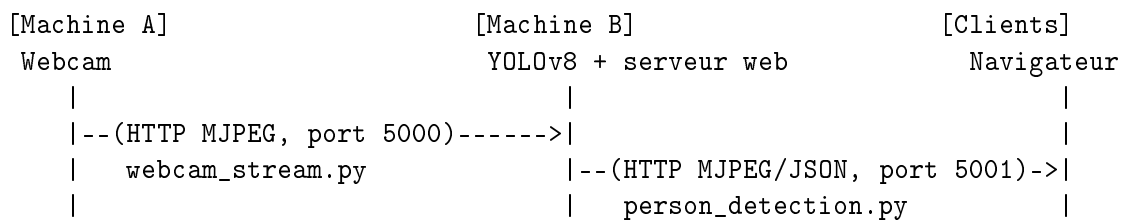
1	Objectif du projet	2
2	Architecture générale	2
3	Technologies utilisées	2
4	Structure des fichiers	2
4.1	webcam_stream.py (Machine A)	3
4.2	person_detection.py (Machine B)	3
5	Mise en réseau	3
6	Problèmes rencontrés et solutions	3
7	Installation et utilisation	3
8	Résultat obtenu	4
9	Pistes d'évolution	4

1 Objectif du projet

Mettre en place un système réparti sur deux machines communiquant sur un réseau local (créé via un point d'accès Wi-Fi partagé depuis un téléphone) :

- une machine capture le flux d'une webcam et le diffuse sur le réseau ;
- une seconde machine récupère ce flux, y applique un modèle d'IA existant (YOLOv8) pour détecter les personnes présentes, et rediffuse à son tour le résultat (vidéo annotée + statistiques) pour qu'il soit consultable depuis n'importe quel appareil du réseau (tablette, téléphone, autre PC).

2 Architecture générale



- **Machine A** : capture la webcam avec OpenCV, encode chaque image en JPEG, et la diffuse en continu via un serveur Flask (flux MJPEG).
- **Machine B** : se connecte au flux de la machine A, fait tourner YOLOv8 sur chaque image pour détecter les personnes, dessine les boîtes de détection, puis republie le résultat (image annotée + compteur de personnes + FPS) via son propre serveur Flask, sous forme de flux vidéo et d'API JSON.
- **Clients** (tablette, téléphone, autre PC) : n'importe quel appareil connecté au même réseau peut ouvrir un navigateur et consulter le résultat, sans rien installer.

Les deux machines et les appareils clients sont connectés au **même point d'accès Wi-Fi**, créé en partageant la connexion d'un téléphone (hotspot).

3 Technologies utilisées

Composant	Rôle
Python	Langage utilisé pour les deux machines
OpenCV (opencv-python)	Capture webcam, lecture de flux, encodage JPEG
Flask	Serveur web léger pour diffuser vidéo et API
Ultralytics YOLOv8	Modèle de détection d'objets pré-entraîné (COCO)
Threading (Python)	Exécuter détection et serveur web en parallèle

Le modèle utilisé est **YOLOv8n** (« nano »), la version la plus légère de YOLOv8, pré-entraînée sur le dataset **COCO** (80 classes d'objets courants). On filtre uniquement la classe 0 = *person*.

4 Structure des fichiers

```
machine_A_streaming/  
  webcam_stream.py    -> capture + diffusion du flux webcam (port 5000)  
  requirements.txt    -> flask, opencv-python
```

```

machine_B_detection/
  person_detection.py  -> reception du flux, detection YOLOv8, serveur web (port 5001)
  requirements.txt     -> opencv-python, ultralytics, flask

README.md              -> guide d'installation et d'utilisation
documentation.tex      -> ce document

```

4.1 webcam_stream.py (Machine A)

Ouvre la webcam locale (`cv2.VideoCapture`), encode chaque image capturée en JPEG, et l'expose via la route Flask `/video_feed` sous forme de flux MJPEG (`multipart/x-mixed-replace`), consommable par n'importe quel client HTTP, y compris un simple navigateur ou `cv2.VideoCapture` sur une autre machine.

4.2 person_detection.py (Machine B)

Fonctionne en deux parties qui tournent en parallèle grâce à un thread :

1. **Boucle de détection** (thread de fond) : lit en continu les images du flux de la machine A, les passe dans YOLOv8, dessine les boîtes autour des personnes détectées, calcule le nombre de personnes et le FPS, et stocke le résultat (image + statistiques) dans une variable partagée protégée par un verrou (`threading.Lock`).
2. **Serveur Flask** (thread principal) : expose trois routes qui lisent cette variable partagée :
 - `/` — page HTML complète avec la vidéo et le compteur mis à jour chaque seconde ;
 - `/video_feed` — flux vidéo annoté seul (format MJPEG) ;
 - `/api/status` — données brutes en JSON (`nb_personnes`, `fps`, `derniere_maj`), utile pour construire d'autres outils par-dessus (tableau de bord, historique, alertes...).

5 Mise en réseau

Le réseau local a été créé via le **partage de connexion (hotspot) d'un téléphone**. Les deux PC s'y connectent comme à un Wi-Fi classique, puis communiquent entre eux via leurs adresses IP locales (plage `10.174.254.x` dans notre cas — cette plage dépend du téléphone utilisé).

Étapes suivies :

1. Activation du hotspot sur le téléphone.
2. Connexion des deux PC au même réseau Wi-Fi.
3. Récupération de l'adresse IP de chaque machine (`ipconfig` sous Windows).
4. Lancement de `webcam_stream.py` sur la machine A, puis de `person_detection.py` sur la machine B en lui donnant l'URL du flux de A.

6 Problèmes rencontrés et solutions

7 Installation et utilisation

```

# Machine A
cd machine_A_streaming
pip install -r requirements.txt
python webcam_stream.py

```

```

# Machine B

```

Problème	Cause	Solution
Confusion entre les IP des deux machines	Le premier <code>ipconfig</code> envoyé était celui de la mauvaise machine	Vérification systématique de quelle machine exécute quelle commande
<code>ping</code> en échec (100% de perte) alors que le réseau fonctionnait	Windows classe les hotspots en profil réseau « Public », qui bloque par défaut les requêtes ICMP (ping) entrantes, même quand la machine est joignable	Test direct du vrai flux HTTP (plus fiable que le ping); possibilité alternative : activer manuellement les règles « Demande d'écho ICMPv4-In » dans le pare-feu Windows avancé
Popup de pare-feu au premier lancement	Windows demande confirmation avant d'autoriser une application à écouter sur le réseau	Autoriser Python à communiquer sur le réseau lors de la popup

```
cd machine_B_detection
pip install -r requirements.txt
python person_detection.py --stream-url http://<IP_MACHINE_A>:5000/video_feed
```

Accès depuis un autre appareil (tablette, téléphone, autre PC) sur le même réseau : http://<IP_MACHINE_B>:5001/

8 Résultat obtenu

- Diffusion webcam fonctionnelle en temps réel sur le réseau local.
- Détection de personnes fonctionnelle (boîtes de détection et comptage).
- Résultat consultable depuis n'importe quel appareil du réseau via navigateur, sans installation nécessaire côté client.
- API JSON disponible pour de futures extensions (tableau de bord, logs, alertes...).

9 Pistes d'évolution

1. **Détecter d'autres objets, sans rien changer techniquement** — YOLOv8 est déjà entraîné sur 80 classes (dataset COCO) : voitures, vélos, sacs, téléphones, animaux, chaises, ordinateurs... Il suffit de changer la liste `classes=[...]` dans `person_detection.py`. Effort : quasi nul.
2. **Suivi et comptage plus intelligent (tracking)** — Actuellement le comptage se fait par image, sans savoir si c'est la même personne d'une image à l'autre. Ultralytics propose du tracking intégré (`model.track()`, avec ByteTrack) : chaque personne reçoit un identifiant unique, ce qui permet un vrai comptage d'entrées/sorties. Effort : modéré.
3. **Zones d'intérêt et alertes** — Définir une zone dans l'image (ex. une porte, une zone interdite) et déclencher une alerte (son, notification, e-mail) si une personne y entre. Effort : modéré.
4. **Historique et statistiques** — Enregistrer chaque détection dans un fichier CSV ou une petite base SQLite (horodatage, nombre de personnes), puis générer un graphique d'affluence dans le temps. Bonne valeur ajoutée pour un rapport de projet.
5. **Détection spécialisée** — Port du masque, casque de sécurité (EPI), détection de chute :

des modèles pré-entraînés spécifiques existent, mais demandent d'intégrer un second modèle YOLO entraîné sur ces classes-là. Effort : plus élevé.

6. **Multi-caméras** — Ajouter une deuxième machine A (deuxième webcam) et afficher les deux flux sur le tableau de bord de la machine B. Effort : modéré, réutilise l'architecture existante.

Remarque : des extensions comme la reconnaissance faciale ou le comptage par âge/genre impliquent des données biométriques, plus sensibles au regard du RGPD — à mentionner avec précaution si le sujet est abordé dans le rapport, bien que cela ne pose pas de problème pour un projet pédagogique de ce type.